

BeSecured

P12 – Personal Cybersecurity Risk Scanner

DETAILED PROJECT PLAN

Project Manager: Mohammedreda El Aroui
Version: 1.0 | April 2026
Duration: 10 Weeks

Group Members

1	Mohammedreda El Aroui
2	Sidi Hamadi
3	Philémon Vittet
4	Abdelrahmn Dahy

1. Project Overview

BeSecured is a 10-week academic project aiming to build a lightweight, user-friendly cybersecurity risk scanner for individuals and small organizations with little to no technical background. The tool will scan users' environments for common vulnerabilities, compute a risk score, and present clear, actionable recommendations through an intuitive dashboard.

This document provides the detailed project plan derived from the Project Charter (v2, April 7 2026). It covers the sprint breakdown, task assignments, risk mitigations, tooling recommendations, and team communication protocols.

2. Detailed Sprint Schedule (10 Weeks)

The project is organized into 10 one-week sprints. Each sprint has a defined phase, key tasks, responsible owners, and an expected deliverable. Orange rows indicate optional features that may be descope if the timeline is at risk.

Week	Phase	Key Tasks	Owner(s)	Deliverable
Week 1	Kick-off & Charter	Finalize charter, set up repo, define DoD, configure tools (Jira/ProjectLibre, GitHub)	PO / SM	Project Charter
Week 2	Project Planning	Detailed backlog creation, story mapping, risk register, architecture decision, tech stack finalized	PO / SM	Project Plan + Backlog
Week 3	System Design	Architecture diagram, module breakdown (Scanner / Risk Engine / Dashboard), DB/data-flow design, UI wireframes	Devs / PO	Design Docs & Wireframes
Week 4	Environment Setup	Dev environment setup, CI/CD pipeline, unit test framework, skeleton codebase, code review workflow	Devs	Working Dev Env + CI
Week 5	Scanner Development	Implement non-intrusive system checks (open ports, weak passwords, software versions, firewall status), scan orchestrator	Devs	Working Scanner Module
Week 6	Risk Engine	Risk scoring algorithm, severity weighting, false-positive validation layer, prioritized recommendation engine	Devs	Risk Score Engine
Week 7	Dashboard & Report	Dashboard UI (risk score display, issue list, remediation steps), PDF/HTML report export, accessibility review	Devs / PO	Dashboard + Report Export

Week	Phase	Key Tasks	Owner(s)	Deliverable
Week 8	AI Integration (optional)	LLM-based plain-language explanation of risks (if timeline allows), fallback to rule-based text if not feasible	Devs	AI Explanation Module
Week 9	Testing & QA	Unit, integration & user acceptance tests; bug fixes; performance benchmarking; privacy audit; test with target users	All team	Test Report + Bug-Free Build
Week 10	Final Delivery	Final demo prep, presentation slides, documentation, code freeze, handoff package, lessons learned	All team	Final Presentation

3. Product Backlog (User Stories)

The following user stories constitute the core product backlog. Stories are prioritized using MoSCoW (Must/Should/Could/Won't). Story points follow a Fibonacci scale.

ID	User Story	Points	Priority	Sprint	Status
US-01	As a user, I can launch the scanner and start a system check with one click	5	Must	W5	To Do
US-02	As a user, I can see a real-time progress bar while the scan runs	3	Should	W5	To Do
US-03	As a user, I receive a cybersecurity risk score (0-100) after each scan	8	Must	W6	To Do
US-04	As a user, I can view a list of detected issues sorted by severity	5	Must	W7	To Do
US-05	As a user, each issue comes with a plain-language explanation and fix steps	8	Must	W7	To Do
US-06	As a user, I can export my scan report as a PDF or HTML file	5	Should	W7	To Do
US-07	As a user, a false-positive validation layer cross-checks results before display	8	Must	W6	To Do
US-08	As a user, I am clearly informed that no personal data is stored or transmitted	3	Must	W5	To Do
US-09	As a user, I can re-run a scan and compare results over time	5	Could	W8	To Do
US-10	As a user, I can get AI-generated plain-language risk explanations (optional)	13	Won't (opt.)	W8	To Do

4. Risk Register & Mitigation Plan

Severity uses the charter scale (5 = Critical, 4 = High, 3 = Medium). Each risk is paired with a concrete mitigation action and a designated owner.

ID	Risk Description	Severity	Level	Mitigation Strategy	Owner
R-01	Difficulty accessing system data (OS restrictions)	4	High	Fallback to limited checks; notify user of reduced accuracy	Devs
R-02	False positives mislead users	3	Med	Add validation layer to cross-check results before display	Devs
R-03	Performance degradation during scan	3	Med	Apply optimization techniques; cap scan scope if needed	Devs
R-04	Data privacy concerns / legal restrictions	5	Critical	No sensitive data stored or transmitted; clear in-app privacy notice	SM / PO
R-05	Team lacks cybersecurity expertise	3	Med	Integrate instructor feedback loops; allocate research time each sprint	All
R-06	Schedule slippage (AI integration scope creep)	4	High	Treat AI integration as optional; descope if W8 is at risk	PO / SM

5. Recommended Tools & Technologies

The following tooling is recommended based on the project's scope, team size, and 10-week timeline. Tools are grouped by category. Each recommendation includes a rationale and a practical usage tip.

Project Management

Tool	Why use it	How to use it in this project
Jira / ProjectLibre	Sprint planning, backlog grooming, issue tracking, burndown charts. ProjectLibre is the free open-source option already mentioned in the charter.	Create one board per sprint. Link each ticket to a user story. Use Jira's roadmap view for stakeholder updates.
GitHub Projects	Kanban board natively integrated with your code repo. Zero extra cost, zero extra login.	Use column labels: Backlog → In Progress → Review → Done. Auto-link PRs to tasks.
Notion / Confluence	Centralized knowledge base for design decisions, meeting notes, and status reports sent to the instructor.	One page per sprint. Embed Jira tickets and architecture diagrams directly.

Version Control & CI/CD

Tool	Why use it	How to use it in this project
GitHub	Source of truth for all code. Supports branch protection, pull request reviews, and Actions for automation.	Enforce: main branch is protected, all changes via PR, minimum 1 review before merge.
GitHub Actions	Free CI/CD pipeline. Run tests automatically on every push, block merges on failures.	Define a workflow: lint → unit tests → integration tests → build artifact.
pre-commit hooks (Husky)	Catch formatting and linting errors before they reach the repo.	Configure eslint / flake8 and run tests locally before each commit.

Development & Security

Tool	Why use it	How to use it in this project
Python 3.x	Ideal for system-level checks (psutil, socket, subprocess). Rich cybersecurity library ecosystem.	Use virtual environments (venv). Pin all dependencies in requirements.txt.
psutil	Cross-platform library for scanning CPU, memory, disk, network connections, and running processes.	Use psutil.net_connections() for port checks; psutil.users() for session auditing.
nmap / python-nmap	Safe, well-documented port and service scanner. Used in 'safe' mode only (-sV, no exploitation).	Wrap nmap calls with a rate-limiter and user-consent prompt before execution.

Tool	Why use it	How to use it in this project
OpenVAS / Greenbone (reference only)	Industry-standard vulnerability scanner – useful as a reference for which checks to implement, but not deployed in the tool.	Review OpenVAS NVT (Network Vulnerability Tests) feed to select appropriate safe checks.

Frontend & Dashboard

Tool	Why use it	How to use it in this project
React + Tailwind CSS	Fast to build, well-documented, great for data-driven dashboards. Vite provides instant HMR.	Component breakdown: ScanButton → ProgressBar → ScoreCard → IssueList → RecommendationPanel.
Chart.js / Recharts	Lightweight charting library to visualize the risk score and issue distribution.	Donut chart for score, bar chart for issue categories, color-coded by severity.
Electron (optional)	Package the web dashboard as a native desktop app (Windows/Mac/Linux) with local system access.	Use only if a desktop installer is required. Adds ~80MB to bundle size.

Testing

Tool	Why use it	How to use it in this project
pytest	Python's standard testing framework. Supports fixtures, parametrize, and coverage reporting.	Aim for ≥80% line coverage on the Scanner and Risk Engine modules.
Jest + React Testing Library	Unit and integration tests for the React frontend.	Test each UI component in isolation; mock the scanner API responses.
Postman / httpie	Manual and automated API testing if a REST layer is added between the scanner and UI.	Export Postman collections as part of the final handoff documentation.
SonarCloud (free for open-source)	Automated code quality and security vulnerability detection in CI.	Connect to GitHub Actions; fix any 'Security Hotspot' findings before final delivery.

Communication

Tool	Why use it	How to use it in this project
Discord / Slack	Daily async communication. Create dedicated channels: #general, #dev, #blockers, #sprint-reviews.	Use #blockers daily. Any unresolved blocker after 24h escalates to SM.
Google Meet / Teams	Weekly sprint reviews with the sponsor and status calls with the instructor.	Record sessions; share a 1-paragraph written summary in Notion within 24h.

Tool	Why use it	How to use it in this project
Figma (free tier)	Collaborative UI design and wireframing. Share mockups with sponsor before development.	Create a shared Figma project with one frame per screen. Link from Notion.

AI Integration (Optional – Week 8)

Tool	Why use it	How to use it in this project
OpenAI API / Claude API (Anthropic)	Generate plain-language explanations of scan findings for non-technical users.	Send structured JSON of detected issues; prompt the model to return bullet-point remediation steps in simple language. Cache responses to avoid repeated API calls.
LangChain (optional)	Framework for chaining prompts, managing context, and structuring LLM interactions.	Use only if multi-step reasoning or retrieval-augmented generation is needed. Keep it simple if the timeline is tight.
Ollama (local LLM fallback)	Run a small LLM locally (e.g., Mistral 7B) to avoid external API costs and data privacy issues.	Package as an optional background service. Fall back to rule-based text if Ollama is unavailable.

6. Communication Plan

An expanded communication plan based on the charter's initial plan, adding Agile ceremonies and a clear escalation path.

Communication	Description	Frequency	Format	Participants
Daily Stand-up	3-question async check-in: What did I do? What will I do? Any blockers?	Daily	Discord / Slack	Full team
Sprint Planning	Define sprint goal, select backlog items, assign tasks for the week	Weekly (Mon)	Video call + Jira board	PO, SM, Devs
Sprint Review	Demo completed features, collect sponsor feedback, update backlog	Weekly (Fri)	Video call	Team + Sponsor
Sprint Retrospective	What went well / could improve / action items for next sprint	Weekly (Fri)	Retro board (EasyRetro)	Full team
Status Report	1-page written summary of progress, blockers, and next steps	Weekly	Document → Notion	Instructor
Issue Escalation	Unresolved blocker escalated from dev to SM within 24 h	Continuous	Discord #blockers	SM + blocked dev
Sponsor Checkpoint	High-level project health update; validate direction and priorities	Bi-weekly	Video call or email	PO + Sponsor

7. Success Criteria

Directly from the charter, reformatted as measurable acceptance criteria for each sprint review.

ID	Criterion	Measurable Target
SC-01	Application stability	≥ 95% of test runs execute without critical errors
SC-02	Risk score generation	100% of valid scans produce a cybersecurity risk score
SC-03	Vulnerability detection rate	≥ 80% of known test-case vulnerabilities are correctly detected
SC-04	User comprehension	≥ 80% of test users rate recommendations as clear and useful

ID	Criterion	Measurable Target
SC-05	Usability	A new user can complete a full scan in under 5 minutes without assistance
SC-06	Schedule	All 10 deliverables delivered within the defined weekly schedule
SC-07	Team velocity	≥ 90% task completion rate per sprint
SC-08	Privacy compliance	Zero sensitive data stored or transmitted outside user's local environment